

Java Unit Testing — JUnit 5 & TestNG

Java Backend Interview — Short Notes

1. Testing Strategy

1.1 Testing Pyramid

Level	Scope	Speed	Cost	Tools	Example
Unit	Single class/method. Dependencies mocked.	Milliseconds	Cheap	JUnit 5, TestNG, Mockito	Test OrderService. calculateTotal() with mocked repo.
Integration	Multiple components together (DB, cache, MQ).	Seconds	Medium	Spring @SpringBootTest, Testcontainers	Test OrderControl ler → Service → real DB.
E2E / Contract	Full application flow via API/UI.	Minutes	Expensive	REST-assured, Selenium, Pact	POST /orders → verify DB + email + Kafka event.

✓ Target: 70% unit, 20% integration, 10% E2E. Fast unit tests give quick feedback. Integration tests catch wiring issues.

1.2 JUnit 5 vs TestNG

Feature	JUnit 5	TestNG
Architecture	Extension model. Core = Platform + Jupiter + Vintage.	Monolithic with plugins.
Annotations	@Test, @BeforeEach, @AfterEach, @BeforeAll, @AfterAll	@Test, @BeforeMethod, @AfterMethod, @BeforeClass, @AfterClass
Parameterized	@ParameterizedTest + @ValueSource/@CsvSource/@MethodSource	@DataProvider + @Test(dataProvider = '...')
Grouping	@Tag('slow'). Filter via maven-surefire.	@Test(groups='regression'). testng.xml for suite config.
Parallel exec	@Execution(CONCURRENT) or junit-platform.properties	Built-in: parallel='methods' in testng.xml. More mature.
Dependencies	Tests must be independent (default). @TestMethodOrder for order.	@Test(dependsOnMethods='loginTest') — explicit dependency.
Assertions	Assertions class. Built-in assertThrows, assertAll.	Assert class. SoftAssert for non-failing multiple assertions.
Spring support	@ExtendWith(SpringExtension.class) or @SpringBootTest	@SpringBootTest works. Less idiomatic with Spring.
Industry adoption	Standard in Spring Boot projects. More popular.	Used in enterprise/legacy projects. Strong in data-driven testing.

2. JUnit 5 — Complete Reference

2.1 Core Annotations

Annotation	Purpose	Notes
------------	---------	-------

@Test	Marks method as test case.	No parameters needed. Replaces JUnit 4 @Test.
@BeforeEach	Run before every test method.	Setup mocks, reset state per test.
@AfterEach	Run after every test method.	Cleanup. Close resources.
@BeforeAll	Run once before all tests in class.	Must be static (or @TestInstance(PER_CLASS)).
@AfterAll	Run once after all tests in class.	Must be static.
@Disabled	Skip this test.	@Disabled('Reason why')
@DisplayName	Human-readable test name in report.	@DisplayName('should throw when price is negative')
@Tag	Group tests by category.	@Tag('slow'), @Tag('regression'). Filter in build.
@Nested	Inner class for related test groups.	Organize: @Nested class WhenUserIsAdmin { ... }
@RepeatedTest	Run test N times.	@RepeatedTest(5) — each run is independent.
@TestMethodOrder	Control execution order.	@TestMethodOrder(OrderAnnotation.class) + @Order(1)
@TestInstance	Control lifecycle.	PER_METHOD (default) or PER_CLASS (share state, non-static @BeforeAll).
@ExtendWith	Register extension (Spring, Mockito, etc).	@ExtendWith(MockitoExtension.class)
@TempDir	Inject temporary directory for file tests.	@TempDir Path tempDir
@Timeout	Fail test if it exceeds time limit.	@Timeout(value=2, unit=SECONDS)

2.2 Assertions

```
import static org.junit.jupiter.api.Assertions.*;

// Basic assertions
assertEquals(expected, actual);
assertNotEquals(unexpected, actual);
assertTrue(condition);
assertFalse(condition);
assertNull(object);
assertNotNull(object);
assertSame(expected, actual); // reference equality
assertArrayEquals(expected, actual);

// Exception testing
IllegalArgumentException ex = assertThrows(
    IllegalArgumentException.class,
    () -> orderService.createOrder(null)
);
assertEquals("Order cannot be null", ex.getMessage());

// Group assertions – all run even if one fails
assertAll("order validation",
    () -> assertEquals("PENDING", order.getStatus()),
    () -> assertNotNull(order.getId()),
```

```

() -> assertTrue(order.getTotal().compareTo(BigDecimal.ZERO) > 0)
);
// Timeout assertion
assertTimeout(Duration.ofSeconds(2), () -> slowService.process());

```

2.3 Parameterized Tests

```

@ParameterizedTest
@CsvSource({
    "10.0, 2, 5.0", // price, qty, expected total
    "25.0, 4, 100.0",
    "0.0, 1, 0.0"
})
void testCalculateTotal(double price, int qty, double expected) {
    assertEquals(expected, service.calculateTotal(price, qty));
}

@ParameterizedTest
@MethodSource("invalidOrderProvider")
void testInvalidOrders(Order order, String expectedMsg) {
    var ex = assertThrows(ValidationException.class, () -> service.validate(order));
    assertEquals(expectedMsg, ex.getMessage());
}

static Stream<Arguments> invalidOrderProvider() {
    return Stream.of(
        Arguments.of(new Order(null, 1), "Product required"),
        Arguments.of(new Order("A", -1), "Quantity must be positive")
    );
}

```

3. Mockito — Mocking Framework

3.1 Core Mockito Concepts

Concept	Annotation / Method	Description	When to Use
Mock	@Mock / mock(Class.class)	Creates fake implementation. All methods return defaults (null, 0, false).	External dependencies: repos, services, HTTP clients.
Spy	@Spy / spy(realObject)	Wraps real object. Calls real methods unless stubbed.	When you need mostly real behavior + stub specific methods.
InjectMocks	@InjectMocks	Creates instance and injects mocks into fields/constructor.	The class under test (the SUT).
Captor	@Captor / ArgumentCaptor	Capture arguments passed to mock methods for assertions.	Verify what exact argument was passed to a mock.
Stub	when(...).thenReturn(...) / doReturn	Define mock method behavior.	Control what collaborators return.
Verify	verify(mock).method(args)	Assert that mock method was called with expected args.	Verify side effects — email sent, event published.

3.2 Mockito in Action

```

@ExtendWith(MockitoExtension.class)
class OrderServiceTest {
    @Mock OrderRepository orderRepo;

```

```

@Mock PaymentService paymentSvc;
@Mock EmailService emailSvc;
@InjectMocks OrderService orderService;

@Test
void shouldCreateOrderSuccessfully() {
    // Arrange
    Order order = new Order("prod-1", 2);
    Order saved = new Order("prod-1", 2); saved.setId(100L);
    when(orderRepo.save(any(Order.class))).thenReturn(saved);
    when(paymentSvc.charge(any())).thenReturn(PaymentResult.SUCCESS);

    // Act
    Order result = orderService.createOrder(order);

    // Assert
    assertEquals(100L, result.getId());
    verify(emailSvc, times(1)).sendConfirmation(saved);
    verify(orderRepo, never()).delete(any()); // never deleted
}

@Test
void shouldThrowWhenPaymentFails() {
    when(paymentSvc.charge(any())).thenThrow(new PaymentException("Declined"));
    assertThrows(PaymentException.class, () -> orderService.createOrder(new Order()));
    verify(orderRepo, never()).save(any()); // rollback – never saved
}

@Test
void shouldCaptureEmailArgument() {
    // Capture exactly what was passed to email service
    ArgumentCaptor<Order> captor = ArgumentCaptor.forClass(Order.class);
    orderService.createOrder(new Order("prod-1", 1));
    verify(emailSvc).sendConfirmation(captor.capture());
    assertEquals("prod-1", captor.getValue().getProductId());
}
}

```

3.3 Argument Matchers

Matcher	Description	Example
any()	Any non-null object of any type.	when(repo.save(any())).thenReturn(saved)
any(Class)	Any non-null object of specified type.	any(Order.class)
anyString()	Any non-null String.	when(svc.find(anyString()))
anyInt()	Any int value.	when(svc.find(anyInt()))
eq(value)	Exact value match.	verify(svc).find(eq("user@email.com"))
isNull()	Match null.	when(svc.find(isNull()))
argThat(pred)	Custom predicate matcher.	argThat(o -> o.getTotal().compareTo(ZERO) > 0)
contains(str)	String containing substring.	verify(log).info(contains("Order created"))

4. TestNG

4.1 TestNG Annotations & Features

Annotation	Description	JUnit 5 Equivalent
@Test	Mark as test. Supports: groups, priority, enabled, timeout, expectedExceptions, dataProvider.	@Test
@BeforeMethod	Run before each test method.	@BeforeEach
@AfterMethod	Run after each test method.	@AfterEach
@BeforeClass	Run once before all tests in class.	@BeforeAll
@AfterClass	Run once after all tests in class.	@AfterAll
@BeforeSuite	Run once before entire test suite (testng.xml).	No direct equivalent
@DataProvider	Supply data to @Test(dataProvider). Returns Object[][].	@MethodSource
@Parameters	Inject parameters from testng.xml into test methods.	No direct equivalent
@Factory	Create multiple instances of test class with different configs.	No direct equivalent
@Listeners	Attach listeners for reporting, screenshots, etc.	@ExtendWith

4.2 TestNG DataProvider & Dependencies

```

@DataProvider(name = "loginData")
public Object[][] loginData() {
    return new Object[][] {
        { "admin", "password", true },
        { "user", "wrong", false },
        { "", "pass", false }
    };
}

@Test(dataProvider = "loginData")
public void testLogin(String user, String pass, boolean expected) {
    assertEquals(authService.login(user, pass), expected);
}

// Test dependency – dependsOnMethods
@Test
public void testUserCreation() { /* ... */ }

@Test(dependsOnMethods = "testUserCreation")
public void testUserLogin() { /* only runs if testUserCreation passes */ }

// SoftAssert – continue after failure
@Test
public void testUserProfile() {
    SoftAssert sa = new SoftAssert();
    sa.assertEquals(user.getName(), "Alice");
    sa.assertNotNull(user.getEmail());
    sa.assertTrue(user.isActive());
    sa.assertAll(); // report all failures at once
}

```

5. Spring Boot Testing

5.1 Testing Slices

Annotation	Loads	Use For	Provides
------------	-------	---------	----------

@SpringBootTest	Full application context.	Integration tests — full stack.	Real beans, real DB (with Testcontainers).
@WebMvcTest	Web layer only (controllers, filters, security).	Controller unit tests — no service/repo.	MockMvc, mocked services.
@DataJpaTest	JPA layer only (repos, entities).	Repository tests — real queries.	In-memory H2 DB (or Testcontainers).
@JsonTest	Jackson serialization.	Test JSON serialization/deserialization.	JacksonTester.
@RestClientTest	REST client (RestTemplate/WebClient).	Test HTTP client code.	MockRestServiceServer.

5.2 MockMvc — Controller Test

```

@SpringBootTest(OrderController.class)
class OrderControllerTest {
    @Autowired MockMvc mockMvc;
    @MockBean OrderService orderService;
    @Autowired ObjectMapper objectMapper;

    @Test
    void createOrder_returnsCreated() throws Exception {
        Order req = new Order("prod-1", 2);
        Order saved = new Order("prod-1", 2); saved.setId(1L);
        when(orderService.createOrder(any())).thenReturn(saved);
        mockMvc.perform(post("/api/orders")
            .contentType(APPLICATION_JSON)
            .content(objectMapper.writeValueAsString(req)))
            .andExpect(status().isCreated())
            .andExpect(jsonPath("$.id").value(1L))
            .andExpect(jsonPath("$.productId").value("prod-1"));
    }
}

```

5.3 Testcontainers — Real DB Integration Test

```

@SpringBootTest
@Testcontainers
class OrderRepositoryIntegrationTest {
    @Container
    static PostgreSQLContainer<?> postgres =
        new PostgreSQLContainer<>("postgres:16-alpine");

    @DynamicPropertySource
    static void props(DynamicPropertyRegistry reg) {
        reg.add("spring.datasource.url", postgres::getJdbcUrl);
        reg.add("spring.datasource.username", postgres::getUsername);
        reg.add("spring.datasource.password", postgres::getPassword);
    }

    @Autowired OrderRepository repo;

    @Test
    void shouldPersistAndRetrieveOrder() {
        Order saved = repo.save(new Order("prod-1", 2));
    }
}

```

```

assertNotNull(saved.getId());
assertEquals("prod-1", repo.findById(saved.getId()).get().getProductId());
}
}

```

6. Interview Questions

Testing Fundamentals

Q1. What is the difference between a mock, stub, and spy?

→ Stub: returns canned responses — no behavior verification. Mock: stub + verifies interactions (was method called? with what args?). Spy: wraps real object, calls real methods unless overridden, can verify. Dummy: placeholder object never actually used. Use mock for collaborators, spy for partial overrides.

Q2. What is test isolation? Why does it matter?

→ Each test runs independently — no shared state, no order dependency. Violated by: static mutable state, database not reset, shared Spring context with modified beans. Broken isolation causes flaky tests (pass alone, fail in suite). Use `@BeforeEach` to reset, `@Transactional` to rollback, or `H2/Testcontainers` per test.

Q3. When should you use `@SpringBootTest` vs `@WebMvcTest`?

→ `@SpringBootTest`: full context — use for integration tests where real beans interact. Slow. `@WebMvcTest`: only web layer loaded — use for controller unit tests where services are `@MockBean`. Fast. `@DataJpaTest`: only JPA layer — use to test repository queries against real DB.

Q4. What is the AAA pattern?

→ Arrange-Act-Assert. Arrange: setup data, configure mocks. Act: call the method under test (single action). Assert: verify the result and interactions. Each test should have exactly one Act. Makes tests readable and focuses each test on one behavior.

Mockito

Q5. When should you use `ArgumentCaptor`?

→ When you need to verify not just that a mock was called but what exact argument was passed — especially for complex objects. Example: verify `EmailService.send()` was called with email containing the correct order ID. More readable than `argThat()` for complex assertions on captured values.

Q6. What is the difference between `@Mock` and `@Spy`?

→ `@Mock`: all methods return defaults unless stubbed (null, 0, false). No real implementation. `@Spy`: calls real implementation unless stubbed. Use spy when you want to override only one method of a real object while keeping others real. Prefer mocks — spies can hide design problems.

Q7. What does `verify()` do? What are common pitfalls?

→ `Verify` checks a mock method was called with specified arguments N times. Pitfalls: (1) Testing internal implementation instead of behavior — brittle. (2) Over-verifying — every `verify()` call = tight coupling to implementation. (3) Missing `verify` on critical side effects (email sent, event published). `Verify` only meaningful interactions.

Advanced

Q8. What is `Testcontainers`? Why use it over `H2`?

→ `Testcontainers` starts real Docker containers for tests — PostgreSQL, Redis, Kafka, MongoDB. `H2` is convenient but doesn't support DB-specific features (JSON columns, window functions, PG-specific functions). `Testcontainers` tests against production-equivalent DB — catch compatibility issues early.

Q9. How do you test async methods (`@Async`)?

→ Options: (1) Use `Awaitility`: `Awaitility.await().atMost(5, SECONDS).until(() -> result.isDone())`. (2) Use `CompletableFuture.get()` with timeout. (3) In Spring tests: `@SpringBootTest` loads `TaskExecutor` — can configure synchronous executor for testing. (4) Test the underlying non-async method directly.

Q10. What is mutation testing? Tools?

→ Inject small code changes (mutations: change `==` to `!=`, remove null check) and verify tests fail. Tests that don't catch mutations are weak. Tool: PIT (Pitest). High mutation score = thorough tests. Use to identify gaps in test coverage beyond line coverage.

